

CASA Evidence — Authentication, Authorization & Access Control

Question: Explain / provide documentation how user authentication and authorization are implemented, what roles and permissions are defined, and how access control rules are enforced when users interact with the application.

Application: Provenance (Next.js App Router monorepo built on the Makerkit "lite" SaaS starter, backed by Supabase — Postgres + GoTrue Auth + Storage).

1. Authentication

Authentication is fully delegated to **Supabase Auth (GoTrue)** — the application never stores passwords or implements its own credential verification.

Mechanism	Where configured	Notes
Email + password	<code>src/config/auth.config.ts</code> (<code>providers.password</code>)	Enabled by default; can be disabled via <code>NEXT_PUBLIC_AUTH_PASSWORD=false</code>
Magic link (passwordless email)	same file (<code>providers.magicLink</code>)	Enabled by default
OAuth — Google	same file (<code>oAuthProviders</code>)	Verified end-to-end
OAuth — Apple	same file	Requires provider config in the Supabase dashboard
CAPTCHA	<code>authConfig.captchaTokenSiteKey</code> → sign-in/sign-up forms	Bot mitigation on public auth forms
Multi-factor authentication (TOTP)	<code>@kit/auth/mfa</code> , <code>src/app/auth/verify/page.tsx</code>	Optional for normal users, mandatory for admins (see §3.3)

1.1 Session lifecycle

bound to the request/response cookies (`createMiddlewareClient`) and calls `supabase.auth.getUser()`. This refreshes the session cookie (access + refresh token) transparently before it reaches any Server Component, Server Action, or Route Handler.

`@supabase/ssr` — never in `localStorage`, never readable by client-side JavaScript.

response: a nonce-based Content-Security-Policy, HSTS (prod + HTTPS), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, and a restrictive `Permissions-Policy`.

- On every request, `src/middleware.ts` builds a Supabase server client
- Session tokens are stored in **httpOnly, secure cookies** managed by
- `middleware.ts` also applies baseline security headers on every

1.2 Account provisioning

credentials/identity.

`kit.new_user_created_setup()` (migration `20241219010757_schema.sql`), automatically inserts a matching row into `public.accounts` for every new `auth.users` row — one account per user, 1:1, populated from the user's name/avatar metadata.

`kit.handle_update_user_email()`, keeps `accounts.email` in sync if the user changes their login email in Supabase Auth.

§2.1) — e.g. an "artist" profile and a "collector" profile can coexist under the same authenticated identity.

- `auth.users` (Supabase-managed) is the source of truth for
- A Postgres trigger, `on_auth_user_created` →
- A second trigger, `on_auth_user_updated` →
- Users layer one or more **profiles** on top of their account (see

1.3 Multi-factor authentication (step-up / AAL)

Assurance Level (AAL) claims.

(`packages/supabase/src/check-requires-mfa.ts`) compares `currentLevel` vs `nextLevel` from `auth.mfa.getAuthenticatorAssuranceLevel()`. If a user has enrolled a factor but the current session is only `aal1`, they are redirected to `/auth/verify` to complete a TOTP challenge (`MultiFactorChallengeContainer`) before continuing.

(`packages/supabase/src/require-user.ts`), so **every** call site that gates a page/action behind "must be signed in" also enforces step-up MFA when applicable.

stricter, fail-closed policy (`src/lib/admin.ts`).

- Implemented with Supabase's built-in TOTP MFA and Authenticator
- `checkRequiresMultiFactorAuthentication()`
- This check is embedded in the shared `requireUser()` helper
- **Admin accounts specifically require MFA** — see §3.3 for the

2. Roles

The app defines three independent, orthogonal role concepts. They are layered on top of Supabase Auth rather than using Makerkit's full team/roles/permissions module, because this deployment uses the **personal-account** schema variant (`20241219010757_schema.sql`): every Supabase user is one `public.accounts` row — there is no separate team-membership/role-assignment table for the base account model. Application-specific roles are instead modeled explicitly in Provenance's own tables, described below.

2.1 Profile role — `public.user_profiles.role`

The primary, user-facing role that drives which product surface a person sees.

`role varchar(20) not null check (role in ('collector', 'artist', 'gallery', 'institution'))`

(from `20250113000000_add_user_profiles.sql`, extended by `20260419000000_add_institution_role.sql`)

Role	Purpose
artist	Manage own artworks, provenance history, CV/grants, exhibitions
collector	Buy/track/own artworks, request certificates
gallery	Represent multiple artists, run exhibitions, manage a storefront/CRM
institution	Museums/foundations — grants, exhibitions, institutional profile

A single `user_id` may hold **multiple profiles** (one `artist` + one `collector` profile is common); a unique index prevents duplicate profiles of the same role for the same user. This role is read by UI/server logic to gate feature areas (dashboards, nav, subscription tier), and is also the `role` recorded on `subscriptions` (billing is per-role/per-profile).

2.2 Gallery team role — `public.gallery_members.role`

A secondary, collaboration-scoped role for multi-user management of a single gallery profile (`20250126000000_add_gallery_members.sql`):

```
role varchar(50) not null default 'member' check (role in ('owner', 'admin', 'member'))
```

Role	Permissions
owner	Full control of the gallery profile; auto-assigned to whoever created the gallery profile (<code>auto_add_gallery_owner</code> trigger)
admin	Can manage members (invite/remove/change roles) and content
member	Can post/manage content, cannot manage membership

2.3 Platform admin flag — `accounts.public_data.admin`

A boolean super-user flag stored in the account's JSON `public_data` column (no dedicated table — "no database changes needed", per the code comment in `src/lib/admin.ts`). This is the only role that grants access to the internal `/admin` operations panel (contacts, outreach, moderation, etc.) and is intentionally **not** combinable with the profile/gallery roles above — it's a flat, single-bit superuser gate rather than a permission matrix.

3. Authorization & Access Control Enforcement

Enforcement follows a **defense-in-depth** model with three independent layers. A request has to pass all three before it can read or write protected data — a bug or omission in the application layer does not, by itself, expose data, because the database layer enforces the same rule independently.

3.1 Layer 1 — Edge / Middleware (`src/middleware.ts`)

Server Components/Actions always see a valid (or explicitly absent) session.

"planet" sub-apps) **before** any authorization decision, and only rewrites to a tenant's route if a published record legitimately exists (looked up via the anon key + RLS-scoped `profile_sites` select — i.e. even this lookup is bound by RLS, not a bypass).

every response, including tenant-rewritten ones.

that is deliberately left to Layer 3 (server components/actions), which have full user/role context; the middleware's matcher also excludes `/api/*`, so every API route is responsible for its own auth check.

- Refreshes the Supabase session on every request so downstream
- Resolves multi-tenant routing (creator subdomains, custom domains,
- Applies the CSP/HSTS/frame/nosniff headers described in §1.1 to
- Does **not** itself decide "is this user allowed to see this page" —

3.2 Layer 2 — Database Row Level Security (Postgres RLS)

This is the authoritative, non-bypassable layer, since the app's Postgres role for normal requests is `authenticated/anon`, not a privileged role.

default privileges on the `public` schema from `anon` (tables, sequences, functions) before any table is created (20241219010757_schema.sql, "Revoke default privileges" section). Nothing is readable/writable until a table `grants` access **and** an RLS policy allows the specific row.

44 migrations / effectively all app tables (accounts, artworks, user_profiles, gallery_members, consignments, exhibitions, subscriptions, notifications, profile_sites, operations_* tables, etc.), totaling **229 create policy statements** across the schema.

`account_id = auth.uid()`.

`authenticated` only where `status = 'verified'` (controlled public exposure, not blanket public read).

require `account_id = auth.uid()` in `using/with check`.

inline, e.g. `gallery_members_read_gallery` allows a gallery's `owner/admin` members to read the full member list, while plain `members` only see their own row (`gallery_members_read_own`).

storage bucket only allows `insert/update/delete` when `split_part(name, '/', 1) = auth.uid()::text` — a user can only write into their own folder prefix.

caller's JWT (`auth.uid()`), this layer holds even if an application bug forgot a check — RLS is what actually stops a horizontal privilege-escalation attempt (User A reading/writing User B's row).

- **Default-deny posture:** the base schema explicitly revokes all
- RLS is enabled (`alter table ... enable row level security`) on
- Standard pattern used almost everywhere (e.g. `artworks`):
- `artworks_read_own` — `select` allowed where
- `artworks_read_public` — `select` allowed for `anon` +
- `artworks_insert` / `artworks_update` / `artworks_delete` — all
- Role-scoped policies compose the same pattern with role checks
- Storage (file) access uses the same principle: e.g. the `artworks`
- Because policies are evaluated by Postgres itself against the

3.3 Layer 3 — Application (Server Components, Server Actions, Route Handlers)

Every protected Server Action / Route Handler obtains a **cookie-bound** Supabase client via `getSupabaseServerClient()` (so all queries it issues are still subject to Layer 2 RLS as that user), then applies one of these shared guards before doing any work:

Guard	File	Used for
<code>requireUser(client)</code>	<code>packages/supabase/src/require-user.ts</code>	Any signed-in-only page/action; also enforces MFA step-up (§1.3)
<code>requireAdmin()</code>	<code>src/lib/admin.ts</code>	Server Component pages under <code>/admin/</code> — redirects to sign-in / home
<code>requireAdminApi()</code>	<code>src/lib/admin.ts</code>	API routes — returns <code>401/403</code> JSON instead of redirecting
<code>requireAdminUser()</code> / <code>requireAdminUserId()</code>	<code>src/lib/admin.ts</code>	Server Actions — throws / returns <code>null</code>
<code>isAdmin(userId)</code>	<code>src/lib/admin.ts</code>	Lightweight boolean check, e.g. <code>GET /api/admin/check</code>

Admin authorization is layered and fail-closed (`src/lib/admin.ts`, `evaluateAdminMfa`):

blocked, redirected/returned as "step-up required."

(`accounts.admin_mfa_grace_deadline`, migration `20260714000000_admin_mfa_grace_period.sql`) allows temporary access with a nudge banner.

already expired, i.e. it **fails closed** rather than grandfathering an admin in indefinitely), the admin is forced to `/settings#security` to enroll MFA before any admin action is permitted.

`step_up_required` (fail closed), never as "allow."

- Must be authenticated (`auth.getUser()`).
- Must have `accounts.public_data.admin === true`.
- MFA assurance is then evaluated:
- If TOTP factors are enrolled but the session is `aal1` →
- If no factors are enrolled, a 7-day grace period
- Once the grace period expires (or was never set — treated as
- Any unexpected error while checking AAL is treated as

Beyond role gates — explicit ownership checks: Server Actions that mutate cross-referenced data additionally re-verify ownership in application code even though RLS already enforces it, e.g. `assertArtworkOwned()` in `src/app/operations/_actions/consignments.ts` re-checks that the artwork's `account_id` matches the caller before allowing a consignment to be created against it. This is intentional belt-and-suspenders: it produces a clean, typed `403`-style error before hitting the database, while RLS remains the last line of defense if this check were ever removed or bypassed.

CSRF / cross-origin protection: Server Actions are POST-only and Next.js enforces same-origin `Origin/Host` header checks on Server Action invocations by default; session cookies are `httpOnly` + `SameSite`-scoped by `@supabase/ssr`, so they are not readable or forgeable from a third-party origin.

4. Putting it together — request walkthrough

Example: an authenticated `gallery` user tries to update a consignment belonging to one of their gallery's artworks. headers, and resolves tenant routing (not applicable here — this is a main-app request).

with a cookie-bound Supabase client:

MFA step-up is pending, the action stops immediately.

explicit application-level ownership check.

additionally filtered by Postgres policies keyed on `auth.uid()`; even if the application check above were skipped, the `consignments/artworks` RLS policies would reject a write against a row the caller doesn't own.

result/errors are logged (`[Operations/consignments] ...`) for traceability.

- **Middleware** refreshes the session cookie, applies security
- **Server Action** (`updateConsignment` in `consignments.ts`) runs
- Calls the shared `requireUser`-style check — if not signed in or
- Calls `assertArtworkOwned(client, userId, artworkId)` — an
- **Database (RLS)** — every `select/update` the action issues is
- Only if all three layers agree does the mutation succeed, and the

5. File map (for auditor traceability)

Concern	File(s)
Session refresh, security headers, tenant routing	<code>src/middleware.ts</code>
Auth provider configuration	<code>src/config/auth.config.ts</code>
Sign-in / sign-up / verify / update-password pages	<code>src/app/auth/sign-in/page.tsx</code> , <code>src/app/auth/verify/page.tsx</code> , <code>src/app/update-password/page.tsx</code>
Shared "require signed-in user" + MFA step-up	<code>makerkit/nextjs-saas-starter-kit-lite/packages/supabase/src/require-user.ts</code> , <code>check-requires-mfa.ts</code>
Admin authorization + MFA enforcement	<code>src/lib/admin.ts</code>
Admin status check API	<code>src/app/api/admin/check/route.ts</code>
Base account schema, provisioning triggers, RLS defaults	<code>makerkit/nextjs-saas-starter-kit-lite/apps/web/supabase/migrations/20241219010757_schema.sql</code>
Profile roles (<code>artist/collector/gallery/institution</code>)	<code>.../migrations/20250113000000_add_user_profiles.sql</code> , <code>20260419000000_add_institution_role.sql</code>
Gallery team roles (<code>owner/admin/member</code>)	<code>.../migrations/20250126000000_add_gallery_members.sql</code>
Admin MFA grace period	<code>.../migrations/20260714000000_admin_mfa_grace_period.sql</code>
Example RLS-protected resource table	<code>.../migrations/20250103000000_create_artworks.sql</code>
Example explicit app-layer ownership check	<code>src/app/operations/_actions/consignments.ts</code>

6. Summary

magic link, Google/Apple OAuth, CAPTCHA on public forms, TOTP MFA), with sessions refreshed server-side via httpOnly cookies on every request.

(*artist/collector/gallery/institution*) for product surface, a *gallery team role* (*owner/admin/member*) for collaboration, and a *platform admin* boolean for the internal ops panel — each defined and constrained at the database level (*check* constraints, dedicated tables).

headers), Postgres Row Level Security (229 policies, default-deny, the authoritative layer), and application-level guards (*requireUser*, *requireAdmin**, plus explicit ownership checks in Server Actions) — with the admin path additionally fail-closed on MFA.

- **Authentication** is handled entirely by Supabase Auth (password,
- **Roles** are explicit and scoped to their purpose: a *profile role*
- **Access control** is enforced redundantly: middleware (session +